Constants

- Syntax:

```
#define constant_name constant_value
```

- As the name implies, variables store values that vary while constants represent fixed values.
- Note that there is no storage when you use constants. Actually, when you compiler your program, the compiler replaces the constant name with the value you defined.
- The pre-processor replaces every occurrence of **`constant_name`** with everything that is to the right of **`constant_name`** in the definition.
  - Note that there is no semicolon at the end of the definition.
- Conventionally, we use names in uppercase for constants.

Enumerated type

- Used to define your own types.
- Syntax:

Text in green is optional

```
enum type_name {
        item_name=constant_int_value, ...
} variable_list;
```

- By default, the value of the first item is 0, and it increases by one for consecutive items. However, you may change the default value by specifying the constant value explicitly.
- Eg:

```
enum boolean {FALSE,TRUE} v1, v2;
enum days {SUN,MON,TUE,WED,THU,FRI,SAT};
enum {one=1,five=5,six,seven,ten=10,eleven} num;
```

Operators

- We will cover the most basic operators in class. More operators will be covered in the labs.
- Assignment operator (=)
  - Note that this is not the "equals" operator. It should be pronounced as "becomes." (Equals is another operator.)
  - The value of the expression on the RHS is assigned (copied) to the LHS.
  - It has right-to-left associativity.

```
a=b=c=10;
```

makes all three variables `10`.

Assignment and type conversion

- When a variable of a narrower type is assigned to a variable of wider type, no problem.
  - Eg:
```
int a=10;
float f;
f=a;
```

- However, there is loss of information in reverse direction.
  - Eg:
```
float f=10.9;
int a;
a=f;
```
